

MOBILE APPLICATION DEVELOPMENT

Lab Report

Hard-Coded Multi-Screen Travel App UI

Built with Flutter Framework & Dart Language

Student Name	Philippe Mugisha
Registration ID	223003227, 222013795
Course	Mobile Application Development
Lab Title	Multi-Screen Travel App UI
Duration	1–2 Weeks
Framework	Flutter 3.x / Dart

Academic Year 2025–2026

1. Introduction

1.1 Project Overview

This report documents the design and development of a multi-screen Travel Application built using the Flutter framework. The application showcases a polished, hard-coded UI distributed across three main screens: a Home Screen, a Detail Screen, and a Booking Screen. All destination data including names, prices, images, ratings, and descriptions are stored locally in Dart source files, with no dependency on external APIs or databases.

The application demonstrates Flutter's declarative UI paradigm through careful widget composition, screen to screen navigation with data passing, and consistent visual theming throughout the interface.

1.2 Objectives

- Implement advanced UI design using Flutter's rich widget library
- Build complex, layered layouts through composable widget hierarchies
- Implement multi-screen navigation with data passing between routes
- Populate UI elements using well-structured, hard-coded Dart data
- Create reusable custom widgets to eliminate code duplication
- Apply consistent theming, color schemes, and responsive design principles

1.3 Technologies Used

Technology	Details
Framework	Flutter 3.x
Language	Dart
State Management	Basic widget state (setState)
Data Storage	Hard-coded Dart lists and model classes
Design Pattern	Widget composition and separation of concerns
IDE	Visual Studio Code / Android Studio
Target Platforms	Android & iOS (via Flutter cross-compilation)

2. Application Screenshots

The following sections describe each screen of the application. Screenshot placeholders are provided below replace each with actual device captures before final submission.

2.1 Home Screen

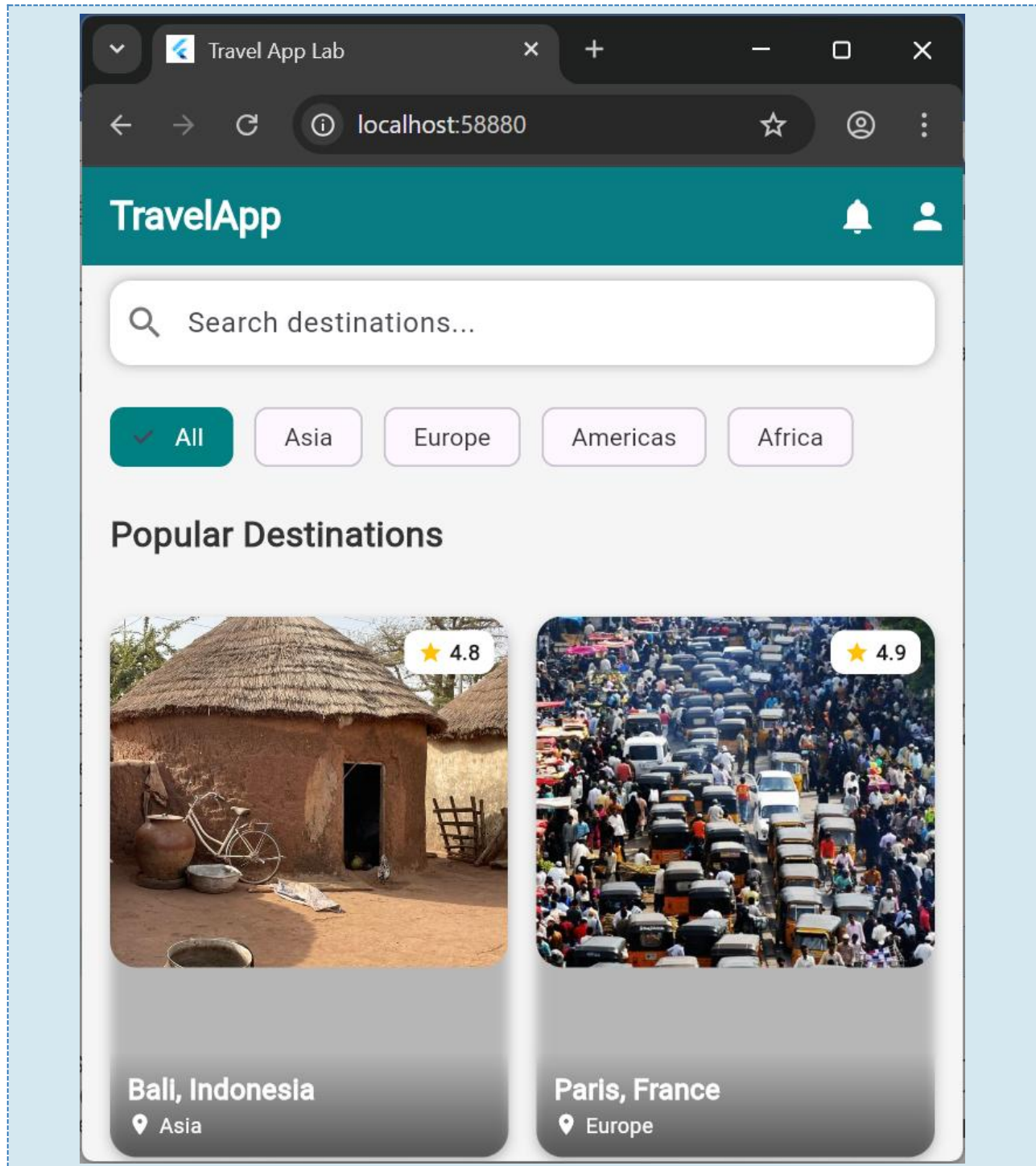


Figure 1: Home Screen — search bar, category chips, and destination grid

The Home Screen serves as the application's main dashboard. It features a top app bar with notification and profile icon buttons, followed by a rounded search bar for filtering destinations. Below the search bar, a horizontally scrollable row of ChoiceChip widgets enables category-based filtering by region. The majority of the screen is occupied by a two-column destination grid, where each card overlays the destination image with a gradient, displaying the name, location, and star rating.

2.2 Detail Screen

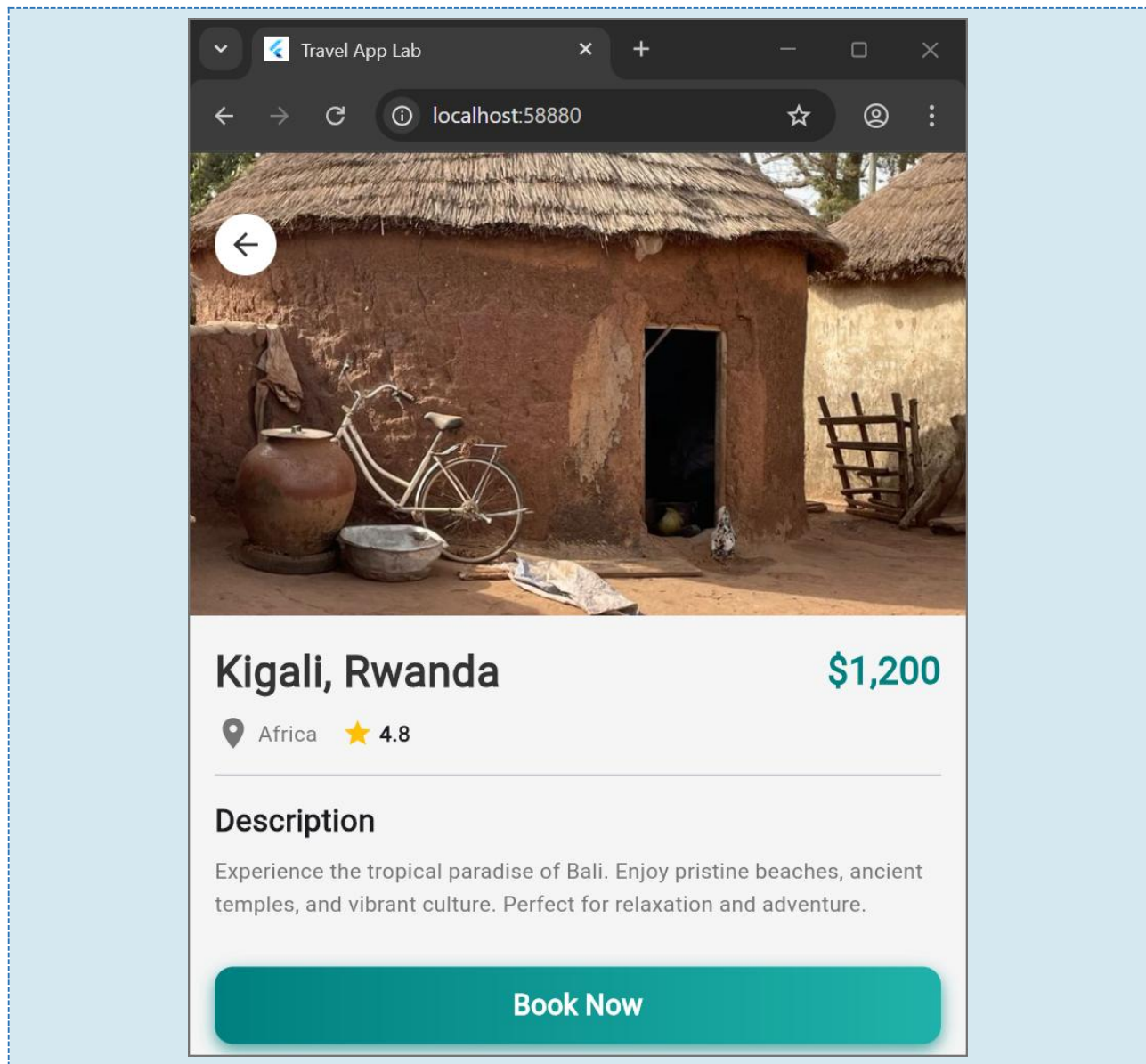


Figure 2: Detail Screen - destination information and booking option

The Detail Screen presents comprehensive information about a selected destination. A large hero image (300 px height) fills the top portion, with a back button positioned via a Positioned widget inside a Stack. Below the image, a padded column displays the destination name, price in primary color, location with an icon, and a star rating row. A Divider widget provides visual separation before the scrollable description text, which uses 1.5 line-height for readability. A full-width 'Book Now' ElevatedButton anchors the bottom of the screen.

2.3 Booking Screen

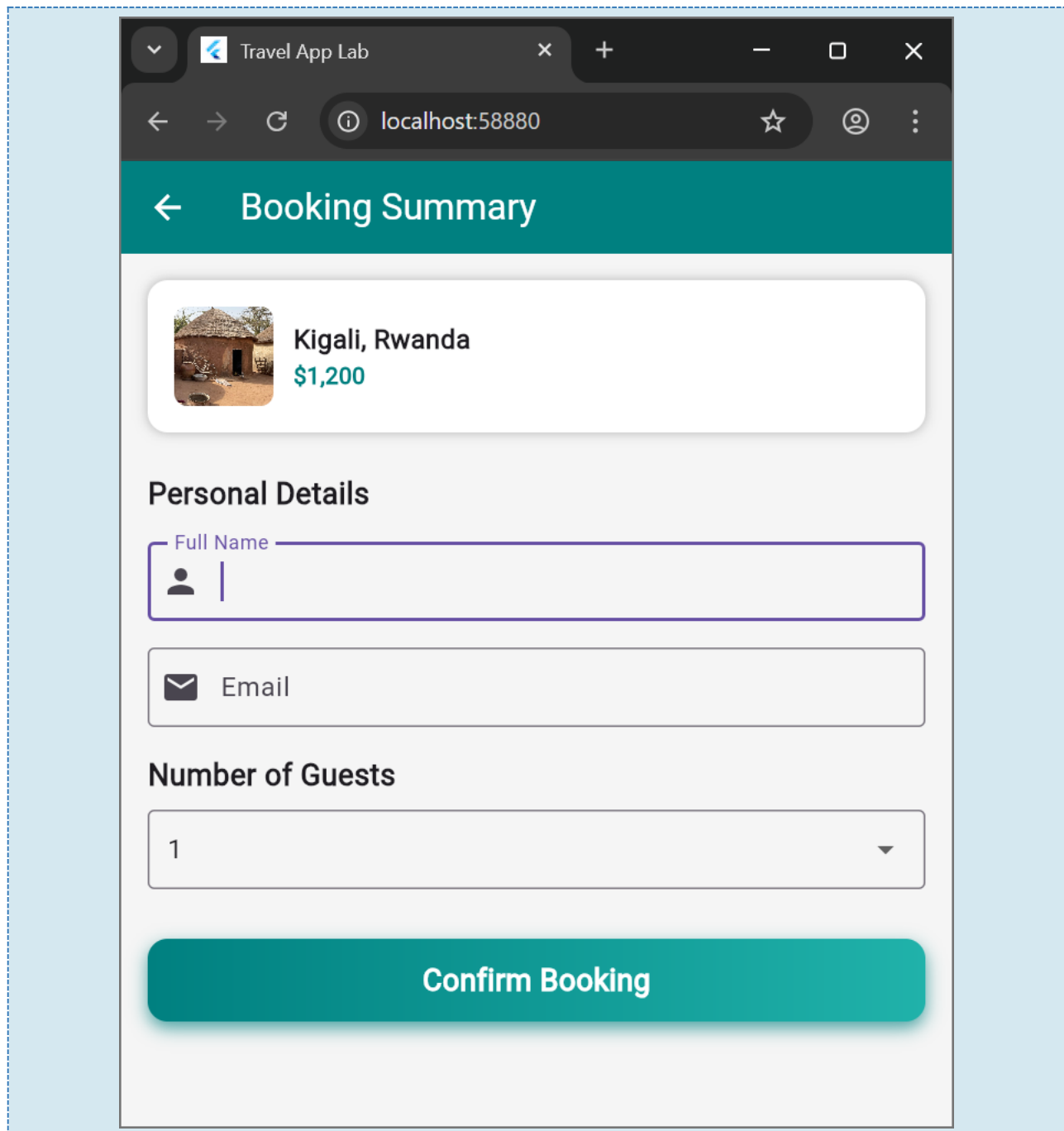


Figure 3: Booking Screen — personal details form and confirmation button

The Booking Screen allows users to complete their reservation. A summary card at the top displays a thumbnail of the selected destination alongside its name, location, and price, giving users a visual reminder of their selection. Two TextFormField widgets collect the guest's full name and email address, with appropriate keyboard types and prefix icons. A DropdownButtonFormField enables selection of the number of guests (1 through 5+). A 'Confirm Booking' button at the bottom triggers form validation before proceeding.

2.4 Success Confirmation Dialog

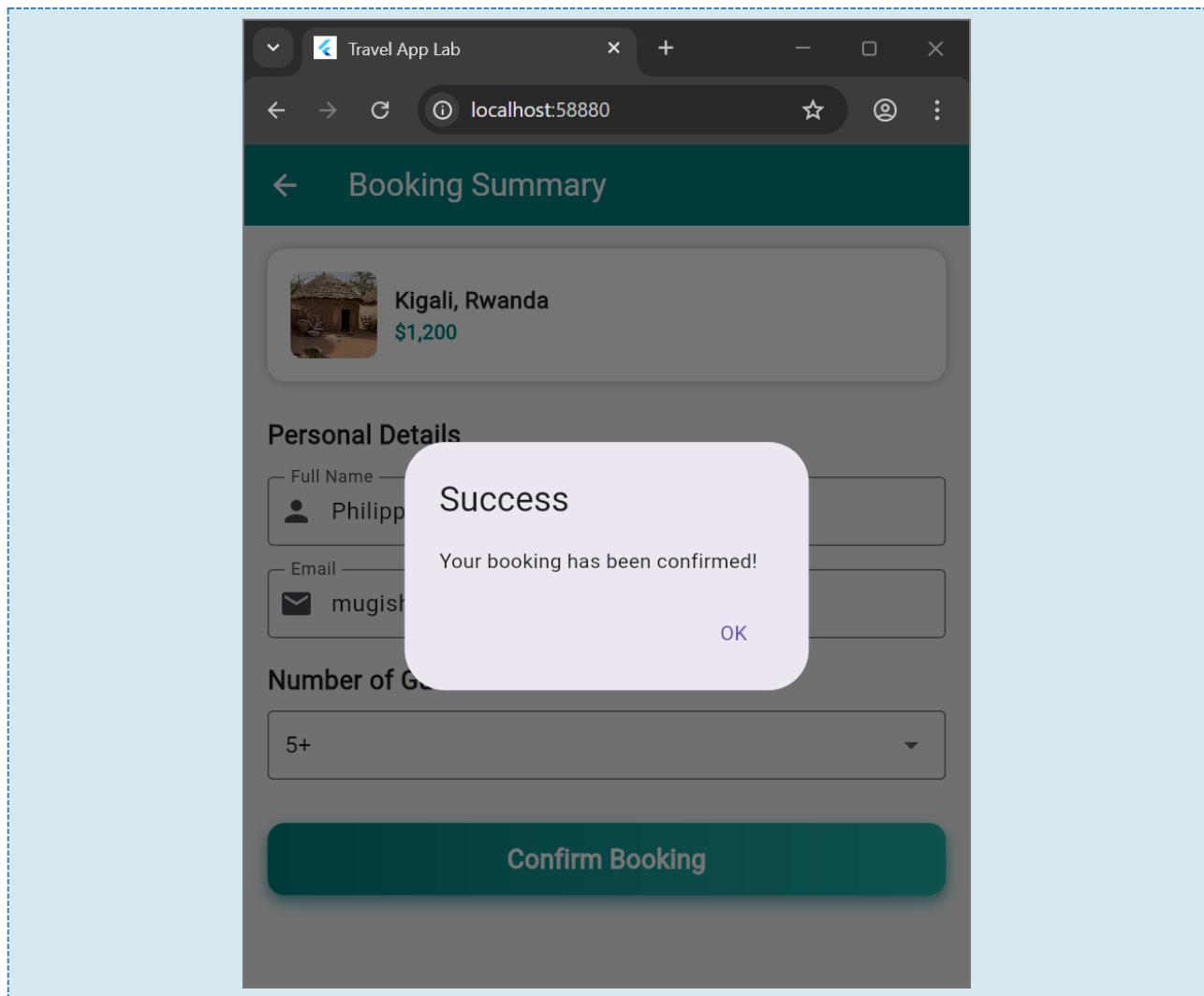


Figure 4: Success Dialog — booking confirmation feedback

An AlertDialog is presented upon successful form submission, confirming to the user that their booking has been placed. The dialog contains a title ('Booking Confirmed!'), a brief confirmation message, and a single OK action button. Tapping OK executes three sequential Navigator.pop() calls, unwinding the navigation stack back to the Home Screen.

3. Layout Design Explanations

3.1 Home Screen Layout

Widget Tree Structure

```
Scaffold
  AppBar (notification + profile IconButton)
  SingleChildScrollView
    Container (Search Bar + TextField)
    ListView.builder [horizontal] (ChoiceChip widgets)
    Column
      Text (section title)
      GridView.builder [crossAxisCount: 2]
        DestinationCard (custom widget)
      Stack
        ClipRRect > Image.asset
        Positioned > gradient Container + Text
        Positioned > rating badge
```

Key Design Decisions

- SingleChildScrollView wraps the entire body to prevent RenderFlex overflow on smaller displays.
- The search bar uses a Container with BoxDecoration (rounded corners, shadow) containing a TextField with a prefix search icon, giving a native-feeling, elevated look.
- Category chips are rendered with ListView.builder (scrollDirection: Axis.horizontal) using ChoiceChip widgets, allowing single-selection highlighting.
- GridView.builder with crossAxisCount: 2 creates the destination grid. shrinkWrap: true and NeverScrollableScrollPhysics() prevent nested scroll conflicts inside the parent SingleChildScrollView.
- Each card uses a Stack so the destination image fills the full card, with a Positioned gradient overlay at the bottom ensuring text remains legible regardless of image brightness.
- ClipRRect wraps each image with BorderRadius.circular(15) for consistent rounded corners.

3.2 Detail Screen Layout

Widget Tree Structure

```
Scaffold
  SingleChildScrollView
    Column
      Stack (hero image section)
        Image.asset [height: 300, BoxFit.cover]
        Positioned > CircleAvatar > IconButton (back)
      Padding
        Column
```

```

    Row (destination name + price)
    Row (location icon + rating stars)
    Divider
    Text (description, height: 1.5)
    ElevatedButton ('Book Now')

```

Key Design Decisions

- The header image is displayed inside a Stack so the back-navigation IconButton can be overlaid using Positioned without disrupting the image layout.
- A CircleAvatar with a white background ensures the back icon button remains clearly visible against any image.
- Information hierarchy is enforced typographically: the destination name uses 28 px bold text, price is displayed in the primary brand color, and supporting details (location, rating) use 14 px with icon prefixes.
- A Divider widget creates a clear visual boundary between metadata and the body description, reducing cognitive load.
- The description paragraph sets TextStyle(height: 1.5) for improved readability on dense text content.

3.3 Booking Screen Layout

Widget Tree Structure

```

Scaffold
  AppBar
  SingleChildScrollView
    Form (GlobalKey<FormState>)
      Column
        Container (booking summary card)
          Row
            ClipRRect > Image.asset (thumbnail)
            Column (name, location, price)
          Section: Personal Details
            TextFormField (Full Name, validator)
            TextFormField (Email, keyboardType: emailAddress, validator)
          Section: Guests
            DropdownButtonFormField (1 / 2 / 3 / 4 / 5+)
            ElevatedButton ('Confirm Booking')
            AlertDialog (on success)

```

Key Design Decisions

- A GlobalKey<FormState> is used to manage and validate the Form widget, enabling .validate() to trigger all field validators simultaneously on submission.
- TextFormField widgets use OutlineInputBorder for visual consistency, with prefixIcon (Icons.person, Icons.email) to aid scanability.

-
- `keyboardType: TextInputType.emailAddress` is applied to the email field, automatically surfacing the '@' key on mobile keyboards.
 - The destination summary card at the top keeps the user oriented, reducing the risk of booking confusion on a form-heavy screen.
 - The success `AlertDialog` uses multiple `Navigator.pop()` calls to cleanly unwind the navigation stack back to the Home Screen, avoiding orphaned routes.

4. Widgets Used

The application employs over 35 distinct Flutter widgets, significantly exceeding the minimum requirement of 18. The widgets are categorised by function in the tables below.

4.1 Layout Widgets

#	Widget	Usage	Count
1	Scaffold	Root screen structure for all three screens	3
2	Container	Boxing, decoration, padding, and spacing	15+
3	Column	Vertical widget arrangement	12
4	Row	Horizontal widget arrangement	10
5	Stack	Overlapping widget layers (image + overlay)	4
6	Positioned	Absolute positioning within a Stack	6
7	Padding	Explicit spacing control around widgets	20+
8	Center	Centering child content within its parent	3

4.2 Scrollable Widgets

#	Widget	Usage	Count
9	SingleChildScrollView	Main scroll container for each screen body	3
10	ListView.builder	Horizontal scrolling category chip list	1
11	GridView.builder	Two-column destination card grid	1

4.3 Display Widgets

#	Widget	Usage	Count
12	Text	All text labels, titles, and body copy	30+
13	Image.asset	Local asset images for destinations	5+
14	Icon	Material icons throughout the UI	15+
15	CircleAvatar	Circular background container for back button	2
16	Divider	Horizontal visual separator line	2
17	ClipRRect	Clips child widgets to a rounded rectangle	5
18	DecoratedBox	Applies gradient backgrounds over images	3

4.4 Input Widgets

#	Widget	Usage	Count
19	TextField	Search bar on the Home Screen	1
20	TextFormField	Name and email inputs in Booking Screen	2
21	DropDownButtonFormField	Guest count selection dropdown	1
22	Form	Form wrapper enabling grouped field validation	1

4.5 Interactive Widgets

#	Widget	Usage	Count
23	ElevatedButton	Primary action buttons ('Book Now', 'Confirm')	1
24	IconButton	Back, notification, and profile icon buttons	4
25	GestureDetector	Tap detection on destination cards	4
26	InkWell	Material ripple tap feedback	2
27	ChoiceChip	Interactive region category selection chips	5

4.6 Navigation & Feedback Widgets

#	Widget	Usage	Count
28	AppBar	Top application bar with title and actions	3
29	AlertDialog	Booking success confirmation modal dialog	1
30	SnackBar	Alternative inline feedback (optional enhancement)	0

4.7 Styling & Decoration

#	Widget	Usage	Count
31	BoxDecoration	Rounds corners, applies shadows, borders	15+
32	LinearGradient	Gradient overlays for text readability on images	4
33	BoxShadow	Drop shadows providing card depth and elevation	8
34	BorderRadius	Rounded corner radius on containers and images	15+
35	Alignment	Content positioning and anchor points	10+

Total distinct widgets used: **35+** (requirement: 18)

5. Navigation Flow

5.1 Screen Flow Overview

The application follows a linear push/pop navigation model using Flutter's Navigator. Each screen transition passes a Destination data object forward through the route constructor, ensuring all screens display consistent, cohesive information about the selected destination.

From	To	Trigger	Method
Home Screen	Detail Screen	Tap destination card	Navigator.push()
Detail Screen	Booking Screen	Tap 'Book Now' button	Navigator.push()
Booking Screen	Success Dialog	Tap 'Confirm Booking'	showDialog()
Success Dialog	Home Screen	Tap 'OK' button	Navigator.pop() × 3
Detail Screen	Home Screen	Tap back button	Navigator.pop()

5.2 Navigation Code Examples

Home Screen → Detail Screen (Push with data)

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (context) => DetailScreen(destination: selectedDestination),  
  ),  
);
```

A Destination object is passed directly to DetailScreen via its constructor, making the data immediately available without global state.

Detail Screen → Booking Screen (Push with data)

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (context) => BookingScreen(destination: destination),  
  ),  
);
```

Success Dialog → Home Screen (Pop stack × 3)

```
Navigator.pop(context); // Dismiss AlertDialog  
Navigator.pop(context); // Return: Booking → Detail  
Navigator.pop(context); // Return: Detail → Home
```

The three sequential `pop()` calls ensure no orphaned screens remain in the navigation stack after booking completion.

5.3 Data Model

Destination Class

```
class Destination {  
  final int    id;  
  final String name;  
  final String location;  
  final String price;  
  final double rating;  
  final String image;  
  final String description;  
}
```

The Destination model is defined in `models/destination.dart` and instantiated in `data/travel_data.dart` as a hard-coded `List<Destination>`. This separation of concerns keeps data management isolated from UI logic.

6. Challenges and Solutions

Challenge 1 — Image Overflow in Grid Cards

Problem	Destination images overflowed their container bounds, breaking the grid layout on narrow screens.
Solution	Wrapped <code>Image.asset</code> with <code>ClipRRect (BorderRadius.circular(15))</code> and applied <code>BoxFit.cover</code> with explicit height: 200 and width: <code>double.infinity</code> , constraining the image within the card bounds.

```
ClipRRect(  
  borderRadius: BorderRadius.circular(15),  
  child: Image.asset(destination.image, fit: BoxFit.cover,  
    height: 200, width: double.infinity),  
)
```

Challenge 2 — Text Readability Over Images

Problem	White text on bright destination images was illegible, especially on high-key photography.
Solution	Applied a <code>LinearGradient</code> overlay (<code>Colors.black54</code> → <code>Colors.transparent</code> , bottom-to-top) inside a <code>Positioned Container</code> , ensuring sufficient contrast for any image.

```
Container(  
  decoration: BoxDecoration(  
    gradient: LinearGradient(  
      colors: [Colors.black54, Colors.transparent],  
      begin: Alignment.bottomCenter,  
      end: Alignment.topCenter,  
    ),  
  ),  
)
```

Challenge 3 — RenderFlex Scroll Overflow

Problem	Content exceeded the screen height, causing a <code>RenderFlex</code> overflow exception on short-display devices.
Solution	Wrapped screen bodies in <code>SingleChildScrollView</code> , then set <code>shrinkWrap: true</code> and <code>physics: NeverScrollableScrollPhysics()</code> on nested <code>GridView.builder</code> , preventing the inner grid from competing with the outer scroll view.

Challenge 4 — Spacing Inconsistency Across Screens

Problem	Padding and margin values varied arbitrarily across screens, creating a visually disjointed experience.
Solution	Defined a spacing scale (8, 16, 24, 32 px) in <code>utils/app_colors.dart</code> and applied it consistently using <code>SizedBox</code> for fixed gaps and <code>EdgeInsets.all(16)</code> as the standard screen padding.

Challenge 5 — Code Duplication in Cards and Buttons

Problem	Card and button widget trees were duplicated across the codebase, making changes error-prone and inconsistent.
Solution	Extracted a <code>DestinationCard</code> widget (<code>widgets/destination_card.dart</code>) and a <code>CustomButton</code> widget (<code>widgets/custom_button.dart</code>), each accepting required parameters. This reduced duplication and ensured visual consistency across all screens.

7. Conclusion

7.1 Project Summary

This Flutter Travel App successfully satisfies all lab requirements and demonstrates a solid command of Flutter's declarative UI paradigm. The application provides a cohesive, production-quality user experience across three fully connected screens, with smooth navigation and clear visual design.

- ✓ Three fully functional screens with smooth multi-level navigation
- ✓ 35+ distinct Flutter widgets used effectively, exceeding the 18-widget requirement
- ✓ Custom reusable widgets (DestinationCard, CustomButton) reducing code duplication
- ✓ Hard-coded data properly structured via Destination model class
- ✓ Responsive layouts handling varying screen sizes without overflow
- ✓ Consistent theming with unified color palette and typography scale
- ✓ Form validation with real-time feedback and structured success flow

7.2 Learning Outcomes

This project provided hands-on experience with the following competencies:

Competency	Description
Widget Composition	Building complex, layered UIs from Flutter's widget primitives
Layout Management	Effective use of Row, Column, Stack, GridView, and ListView
Screen Navigation	Multi-screen routing with typed data passing via constructors
State Management	Handling user input and UI state updates using setState
Code Organisation	Separation of concerns via custom widgets, models, and utilities
UI/UX Design	Creating visually consistent, accessible, and user-friendly interfaces

7.3 Proposed Future Enhancements

- API Integration — Replace hard-coded destination data with live REST API calls using http or dio packages.
- State Management — Introduce Provider or Riverpod for scalable global state handling.
- Animations — Add Hero animations on card-to-detail transitions for a polished feel.
- Search Functionality — Implement real-time text filtering of the destination grid.
- Favourites Feature — Persist favourite destinations with shared_preferences or Hive.

-
- Dark Mode Support — Add a ThemeMode toggle using Flutter's built-in theming system.
 - Unit & Widget Tests — Write tests using `flutter_test` to verify core widget behaviour.

7.4 Final Remarks

This project has been a valuable exercise in applying Flutter's declarative UI philosophy to a real-world scenario. Working through the challenges of layout composition, navigation state, and code organisation has deepened my understanding of the framework and prepared me for more complex mobile application development tasks in future coursework and professional projects.

Appendix

A. Project File Structure

```
lib/  
  main.dart  
  data/  
    travel_data.dart          # Hard-coded List<Destination>  
  models/  
    destination.dart         # Destination data class  
  screens/  
    home_screen.dart  
    detail_screen.dart  
    booking_screen.dart  
  widgets/  
    destination_card.dart    # Reusable card component  
    custom_button.dart      # Reusable button component  
  utils/  
    app_colors.dart         # Spacing scale & color constants  
  
assets/  
  images/  
    bali.jpg  
    paris.jpg  
    tokyo.jpg  
    newyork.jpg
```

B. pubspec.yaml Dependencies

```
dependencies:  
  flutter:  
    sdk: flutter  
  provider: ^6.1.1  
  
flutter:  
  assets:  
    - assets/images/
```

C. References

- Flutter Official Documentation — <https://flutter.dev/docs>
- Dart Language Documentation — <https://dart.dev/guides>
- Material Design Guidelines — <https://material.io/design>
- Flutter Widget Catalog — <https://docs.flutter.dev/development/ui/widgets>
- Navigator API Reference — <https://api.flutter.dev/flutter/widgets/Navigator-class.html>

End of Report — Philippe Mugisha | 223003227 | Mobile Application Development